

LAPORAN PRAKTIKUM
PRAKTIKUM PENGUJIAN PERANGKAT LUNAK
PERTEMUAN 10
PAGE OBJECT MODEL (POM)



Disusun oleh:

Nama : Januarsyah Akbar
NIM : 24/535846/SV/24314
Kelas : PL4B2
Dosen Pengampu : Divi Galih Prasetyo Putri, S.Kom., M.Kom., Ph.D.

PROGRAM STUDI D-IV
TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
May 12, 2026

Daftar Isi

Daftar Isi	2
Tujuan Praktikum	3
Hasil dan Pembahasan	4
2.1 Latihan (POM pada Pencarian Bing)	4
2.1.1 Kelas <code>SearchPage</code>	4
2.1.2 Kelas <code>ResultPage</code>	4
2.1.3 Tahap Pengujian (<code>TestPage</code>)	5
2.2 Tugas (Centralized Locators dan <code>BasePage</code>)	6
2.2.1 Sentralisasi Locator (<code>SauceLocators</code>)	6
2.2.2 Implementasi <code>BasePage</code>	7
2.2.3 Penerapan Turunan (<code>LoginPage</code> & <code>InventoryPage</code>)	8
2.2.4 Tahap Pengujian (<code>LoginTest</code>)	8
Kesimpulan	11

Tujuan Praktikum

1. Mahasiswa mampu memahami konsep *Page Object Model* (POM).
2. Mahasiswa mampu mengimplementasikan POM dalam skenario pengujian perangkat lunak di situs yang riil.

Hasil dan Pembahasan

2.1 Latihan (POM pada Pencarian Bing)

Pada bagian latihan ini, kita mengimplementasikan *Page Object Model* (POM) untuk memisahkan logika skrip uji (*test script*) dengan manajemen akses elemen web. Skenario yang digunakan adalah melakukan pencarian "janu ganteng" pada mesin pencari Bing, di mana strukturnya dibagi menjadi kelas *Page* dan kelas Pengujian.

2.1.1 Kelas SearchPage

```
1 package latihan;
2
3 import org.openqa.selenium.By;
4 import org.openqa.selenium.Keys;
5 import org.openqa.selenium.WebDriver;
6
7 public class SearchPage {
8     private WebDriver driver;
9     private By searchBox = By.name("q");
10
11     public SearchPage(WebDriver driver) {
12         this.driver = driver;
13     }
14
15     public void enterSearchQuery(String query) {
16         driver.findElement(searchBox).sendKeys(query);
17     }
18
19     public void submitSearch() {
20         driver.findElement(searchBox).sendKeys(Keys.ENTER);
21     }
22 }
```

Listing .1: Implementasi SearchPage.java

Kelas ini berisi inisialisasi *locator* (seperti elemen kotak pencarian / `searchBox`) dan memberikan *method* fungsionalitas bagi halaman utama (beranda pencarian) untuk menerima input dan menekan *Enter*.

2.1.2 Kelas ResultPage

```
1 package latihan;
2
3 import org.openqa.selenium.WebDriver;
```

```

4
5 public class ResultPage {
6     private WebDriver driver;
7
8     public ResultPage(WebDriver driver) {
9         this.driver = driver;
10    }
11
12    public String getTitle() {
13        return driver.getTitle();
14    }
15 }

```

Listing .2: Implementasi ResultPage.java

Kelas halaman kedua ini merepresentasikan rupa halaman interaksi ketika Bing telah menampilkan hasil. Skrip memusatkan pengaksesan elemen, yaitu pengambilan judul situs menggunakan metode getTitle().

2.1.3 Tahap Pengujian (TestPage)

```

1 package latihan;
2
3 import org.junit.jupiter.api.AfterEach;
4 import org.junit.jupiter.api.BeforeEach;
5 import org.junit.jupiter.api.Test;
6 import org.openqa.selenium.WebDriver;
7 import org.openqa.selenium.safari.SafariDriver;
8
9 import static org.junit.jupiter.api.Assertions.assertEquals;
10
11 public class TestPage {
12     private WebDriver driver;
13     private SearchPage searchPage;
14     private ResultPage resultPage;
15
16     @BeforeEach
17     public void setUp() {
18         driver = new SafariDriver();
19         driver.manage().window().maximize();
20         driver.get("https://www.bing.com/");
21
22         searchPage = new SearchPage(driver);
23         resultPage = new ResultPage(driver);
24     }
25
26     @Test
27     public void testBingSearch() {
28         String query = "janu ganteng";
29         searchPage.enterSearchQuery(query);
30         searchPage.submitSearch();
31
32         try {
33             Thread.sleep(2000);
34         } catch (InterruptedException e) {
35             e.printStackTrace();
36         }
37     }

```

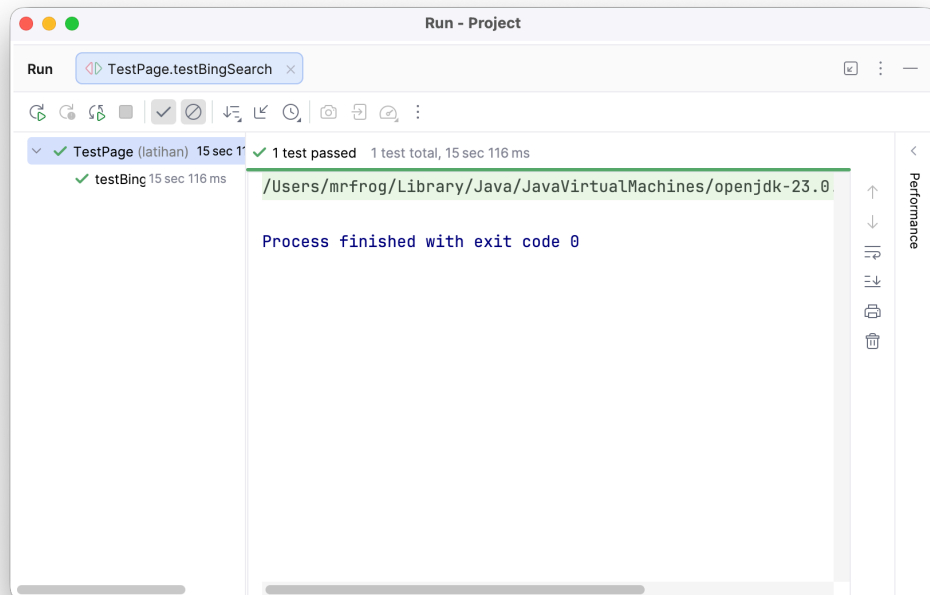
```

38     assertEquals("janu ganteng - Search", resultPage.getTitle());
39 }
40
41 @AfterEach
42 public void tearDown() {
43     if (driver != null) {
44         driver.quit();
45     }
46 }
47 }

```

Listing .3: Implementasi TestPage.java

Blok TestPage berlaku murni sebagai manajer jalannya pengujian. WebDriver dieksekusi di MacOS menggunakan SafariDriver. Seluruh kode *boilerplate* terkait identifikasi *tag* HTML tidak lagi berserakan berkat model POM, sehingga asersi akhir cukup memeriksa properti *title* halaman.



Gambar 1: Bukti Hasil Eksekusi Selenium Latihan POM

2.2 Tugas (Centralized Locators dan BasePage)

Bagian tugas memoles POM jauh lebih mendalam lewat pola desain sentralisasi *locators* dan pendelegasian interaksi umum ke kelas leluhur yang dinamakan BasePage. Situs referensi sasaran bertransformasi ke portal "SauceDemo".

2.2.1 Sentralisasi Locator (SauceLocators)

```

1 package tugas;
2
3 import org.openqa.selenium.By;
4

```

```

5 public class SauceLocators {
6     // Login Page Locators
7     public static final By USERNAME_FIELD = By.id("user-name");
8     public static final By PASSWORD_FIELD = By.id("password");
9     public static final By LOGIN_BUTTON = By.id("login-button");
10
11     // Inventory Page Locators
12     public static final By TITLE_HEADER = By.className("title");
13 }

```

Listing .4: Daftar Sentralisasi Locator pada SauceLocators.java

Seluruh struktur selektor DOM diinsulasi secara global menjadi sekumpulan *static field* berstatus *final*. Ini bertujuan menyatukan poin perbaikan, apabila terjadi evolusi perombakan desain di antarmuka SauceDemo.

2.2.2 Implementasi BasePage

```

1 package tugas;
2
3 import org.openqa.selenium.By;
4 import org.openqa.selenium.WebDriver;
5 import org.openqa.selenium.WebElement;
6 import org.openqa.selenium.support.ui.ExpectedConditions;
7 import org.openqa.selenium.support.ui.WebDriverWait;
8 import java.time.Duration;
9
10 public class BasePage {
11     protected WebDriver driver;
12     protected WebDriverWait wait;
13
14     public BasePage(WebDriver driver) {
15         this.driver = driver;
16         this.wait = new WebDriverWait(driver, Duration.ofSeconds(10));
17     }
18
19     public void click(By locator) {
20         wait.until(ExpectedConditions.elementToBeClickable(locator)).click();
21     }
22
23     public void type(By locator, String text) {
24         WebElement element = wait.until(ExpectedConditions.
25 ↪ visibilityOfElementLocated(locator));
26         element.clear();
27         element.sendKeys(text);
28     }
29
30     public String getText(By locator) {
31         ↪ return wait.until(ExpectedConditions.visibilityOfElementLocated(locator)).
32 ↪ getText();
33     }
34 }

```

Listing .5: Kelas Relasi Utama BasePage.java

BasePage mengadopsi mekanisme *Explicit Wait* terselubung untuk segala operasi inti (*click*, *type*, *getText*). Setiap anak turunan halaman dipastikan menurunkan keamanan stabilitas muatan *Dynamic DOM* secara bebas.

2.2.3 Penerapan Turunan (LoginPage & InventoryPage)

```
1 package tugas;
2
3 import org.openqa.selenium.WebDriver;
4
5 public class LoginPage extends BasePage {
6     public LoginPage(WebDriver driver) {
7         super(driver);
8     }
9
10    public void login(String username, String password) {
11        type(SauceLocators.USERNAME_FIELD, username);
12        type(SauceLocators.PASSWORD_FIELD, password);
13        click(SauceLocators.LOGIN_BUTTON);
14    }
15 }
16
17 // -----
18
19 package tugas;
20
21 import org.openqa.selenium.WebDriver;
22
23 public class InventoryPage extends BasePage {
24     public InventoryPage(WebDriver driver) {
25         super(driver);
26     }
27
28     public String getPageTitle() {
29         return getText(SauceLocators.TITLE_HEADER);
30     }
31 }
```

Listing .6: Kelas LoginPage.java dan InventoryPage.java

Kelas *Page Object* turunan secara signifikan terlihat ringkas dibandingkan versi non-POM konvensional. Variasi parameter (*username* dan *password*) dilontarkan sebagai sekumpulan operan fungsi menuju lapisan pewarisnya (*BasePage*).

2.2.4 Tahap Pengujian (LoginTest)

```
1 package tugas;
2
3 import org.junit.jupiter.api.AfterEach;
4 import org.junit.jupiter.api.BeforeEach;
5 import org.junit.jupiter.api.Test;
6 import org.openqa.selenium.WebDriver;
7 import org.openqa.selenium.safari.SafariDriver;
8
9 import static org.junit.jupiter.api.Assertions.assertEquals;
10
11 public class LoginTest {
12     private WebDriver driver;
13     private LoginPage loginPage;
14     private InventoryPage inventoryPage;
15
16     @BeforeEach
```

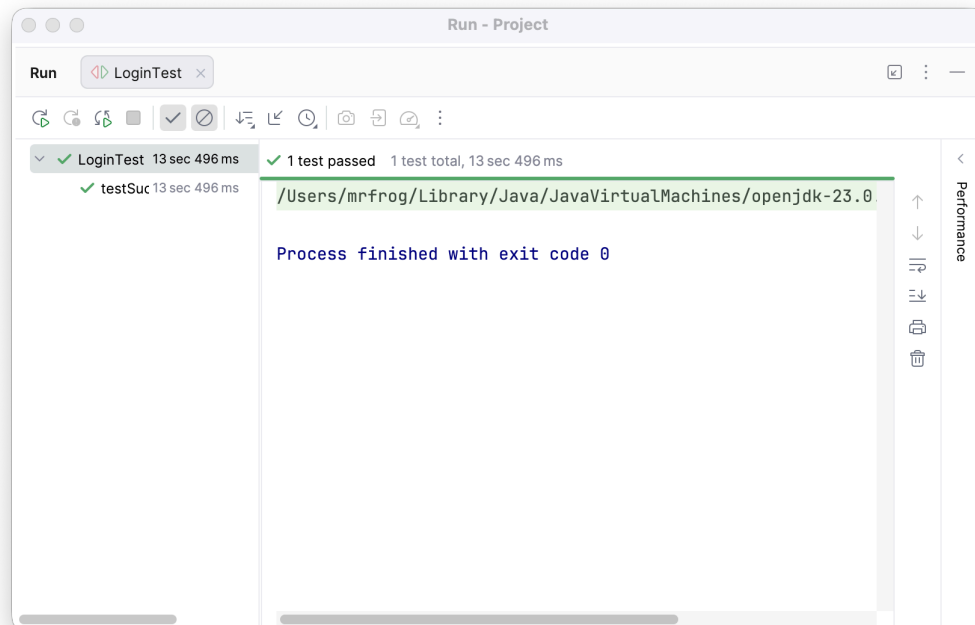


```

17 public void setUp() {
18     driver = new SafariDriver();
19     driver.manage().window().maximize();
20     driver.get("https://www.saucedemo.com/");
21
22     loginPage = new LoginPage(driver);
23     inventoryPage = new InventoryPage(driver);
24 }
25
26 @Test
27 public void testSuccessfulLogin() {
28     loginPage.login("standard_user", "secret_sauce");
29
30     String expectedTitle = "Products";
31     assertEquals(expectedTitle, inventoryPage.getPageTitle());
32 }
33
34 @AfterEach
35 public void tearDown() {
36     if (driver != null) {
37         driver.quit();
38     }
39 }
40 }

```

Listing .7: Hasil Pengujian Evaluasi Modularitas POM pada LoginTest.java



Gambar 2: Bukti Hasil Eksekusi Selenium Tugas POM Berkelanjutan

Repository

Akses kode lengkap untuk praktikum Pertemuan 10 ini dapat ditinjau dan ditarik versinya melalui GitHub Repository terkait yang berisikan struktur modular sistem.

Kesimpulan

Berdasarkan praktikum yang telah diinisiasi pada Pertemuan 10, adopsi dari pola arsitektur *Page Object Model* (POM) secara masif mentransformasi tatanan pengkodean *Quality Assurance* menjadi rapi, kohesif, dan skalabel. Modifikasi skrip pengujian memisahkan elemen manipulatif *web browser* ke dalam domain *Page Object* tunggal, meminimalisir secara drastis redundansi tumpukan pendefinisian `WebElement` berulang.

Dalam skala proyek yang meluas, utilisasi skema *Centralized Locators* berdampingan erat dengan generalitas interaksi pada `BasePage`. Struktur penyelesaian ini memastikan apabila antarmuka visual sebuah aplikasi mengalami restrukturisasi format di level rekayasa front-end, perawatan (*maintenance code*) Selenium cukup terpusat pada satu lokasi deklarasi `By selector`-nya semata tanpa mencemari stabilitas rentetan instansiasi test spesifik lainnya.